EE473 Final Project: Energy-Aware Edge Scheduling with Reinforcement Learning

This notebook is the final report version for submission. It focuses on the core setup, key code, and main experimental results.

1) Problem Setup

We model a single-device scheduler with three actions: `low`, `medium`, `high`. At each step, workload arrives from trace data, the action determines service/energy, and queue/battery are updated.

Reward: `r_t = -(alpha_energy * energy_t + beta_latency * latency_t + gamma_miss * miss_t)`

Default weights: `(alpha, beta, gamma) = (1.0, 0.6, 2.0)`.

2) Dataset and Evaluation Protocol

- Data: Google cluster task-event trace (120 shards)
- Preprocess: aggregate submit events, normalize workload, split train/test
- Episode length: `288`
- Main protocol: `20` train episodes, `6` non-overlap test episodes
- Methods: heuristic baselines, tabular Q-learning, linear approximation Q-learning
- Reporting: 5 seeds with mean/std

3) Environment Sanity Check

Load processed trace data and run one sample episode to verify state transition and reward outputs.

In [1]:
```
from pathlib import Path
import subprocess
import json

ROOT = Path.cwd().resolve().parent
assert (ROOT / 'scripts').exists(), f'Expected repo root at {ROOT}'
print('ROOT =', ROOT)
```

```
ROOT = /Users/zilongzeng/Documents/EE473final project
```

In [2]:
```
import sys
import numpy as np
```

```python
import matplotlib.pyplot as plt
from pathlib import Path

# Add src/ to path so we can import project modules directly
SRC = ROOT / 'src'
if str(SRC) not in sys.path:
    sys.path.insert(0, str(SRC))

from config import DEFAULT_ENV_CONFIG
from data_loader import load_workload_trace, build_episodes
from env import EnergySchedulingEnv

# ── Load workload trace ──────────────────────────────────────────────────
TRACE_PATH = ROOT / 'data/processed/workload_trace_120.csv'
EPISODE_LENGTH = 288
STRIDE = 288

train_series = load_workload_trace(TRACE_PATH, split='train')
test_series  = load_workload_trace(TRACE_PATH, split='test')

train_episodes = build_episodes(train_series, EPISODE_LENGTH, stride=STRIDE)
test_episodes  = build_episodes(test_series,  EPISODE_LENGTH, stride=STRIDE)

print(f"Train episodes : {len(train_episodes)}  ×  {len(train_episodes[0])}
print(f"Test  episodes : {len(test_episodes)}  ×  {len(test_episodes[0])} st
print(f"Workload range : [{min(train_series):.3f}, {max(train_series):.3f}]"
```

```
Train episodes : 20  ×  288 steps
Test  episodes : 6  ×  288 steps
Workload range : [0.000, 1.000]
```

In [3]:

```python
# Quick sanity stats for one training episode (no extra plot in report PDF)
ep0 = np.array(train_episodes[0], dtype=float)
print('Episode 0 workload stats:')
print(f"  mean={ep0.mean():.4f}, std={ep0.std():.4f}, min={ep0.min():.4f}, m
```

```
Episode 0 workload stats:
  mean=0.0902, std=0.1681, min=0.0000, max=1.0000
```

In [4]:

```python
# Run one short random-policy rollout and print compact diagnostics
rng = np.random.default_rng(42)
env = EnergySchedulingEnv(train_episodes[0], config=DEFAULT_ENV_CONFIG)
obs, info = env.reset()
ret = 0.0
for _ in range(288):
    action = int(rng.integers(0, 3))
    obs, reward, done, info = env.step(action)
    ret += reward
    if done:
        break
print(f"Random policy episode return: {ret:.3f}")
print(f"Final queue={info['queue']:.3f}, final battery={info['battery']:.3f}
```

```
Random policy episode return: -151.711
Final queue=0.000, final battery=0.000
```

4) Baseline Evaluation

Evaluate fixed policies (`always_low` , `always_medium` , `always_high` , threshold) on held-out test episodes.

In [5]:
```
from baselines import (
    always_low_policy, always_medium_policy, always_high_policy,
    threshold_policy_factory, evaluate_policy,
)

baseline_policies = {
    'always_low':    always_low_policy,
    'always_medium': always_medium_policy,
    'always_high':   always_high_policy,
    'threshold(w=0.60)': threshold_policy_factory(workload_threshold=0.60),
    'threshold(w=0.80)': threshold_policy_factory(workload_threshold=0.80),
}

print(f"{'Policy':<22}  {'Avg Return':>10}  {'Energy':>7}  {'Latency':>8}  {
print('-' * 65)
baseline_results = {}
for name, policy in baseline_policies.items():
    m = evaluate_policy(test_episodes, policy, config=DEFAULT_ENV_CONFIG)
    baseline_results[name] = m
    print(f"{name:<22}  {m['avg_episode_return']:>10.3f}  "
          f"{m['avg_step_energy']:>7.4f}  {m['avg_step_latency']:>8.4f}  "
          f"{m['miss_rate']:>9.4f}")
```

```
Policy                   Avg Return   Energy   Latency  Miss Rate
-----------------------------------------------------------------
always_low                   -57.289   0.1800    0.0315     0.0000
always_medium               -121.749   0.4200    0.0046     0.0000
always_high                 -244.800   0.8500    0.0000     0.0000
threshold(w=0.60)            -58.512   0.2022    0.0016     0.0000
threshold(w=0.80)            -57.390   0.1951    0.0069     0.0000
```

5) RL Training (Single-Seed Demonstration)

Run tabular and linear-approximation Q-learning once to show training behavior. Final claims are based on saved multi-seed artifacts.

In [6]:
```
import time
from q_learning import QLearningConfig, train_tabular_q_learning

# ── Tabular Q-learning (300 epochs, single seed) ──────────────────────
tabular_cfg = QLearningConfig(
    num_epochs=300,
    alpha=0.15,
    gamma=0.98,
    epsilon_start=0.30,
    epsilon_end=0.02,
    epsilon_decay=0.995,
```

```
    eval_every=5,
    seed=42,
)

t0 = time.perf_counter()
tabular_result = train_tabular_q_learning(
    train_episodes=train_episodes,
    test_episodes=test_episodes,
    env_config=DEFAULT_ENV_CONFIG,
    algo_config=tabular_cfg,
)
tabular_time = time.perf_counter() - t0

print(f"Tabular Q-learning  —  training time: {tabular_time:.2f} s")
bm = tabular_result['best_test_metrics']
print(f"  Best checkpoint (epoch {int(tabular_result['best_epoch'])}):")
print(f"    avg_return = {bm['avg_episode_return']:.4f}")
print(f"    energy     = {bm['avg_step_energy']:.4f}")
print(f"    latency    = {bm['avg_step_latency']:.4f}")
print(f"    miss_rate  = {bm['miss_rate']:.4f}")
```

```
Tabular Q-learning  —  training time: 6.72 s
  Best checkpoint (epoch 115):
    avg_return = -56.2166
    energy     = 0.1878
    latency    = 0.0123
    miss_rate  = 0.0000
```

In [7]:

```
from approx_q import ApproxQLearningConfig, train_linear_approx_q_learning

# —— Linear Approximation Q-learning (400 epochs, single seed) ————————————
approx_cfg = ApproxQLearningConfig(
    num_epochs=400,
    alpha=0.02,
    gamma=0.98,
    epsilon_start=0.30,
    epsilon_end=0.02,
    epsilon_decay=0.997,
    eval_every=5,
    seed=42,
)

t0 = time.perf_counter()
approx_result = train_linear_approx_q_learning(
    train_episodes=train_episodes,
    test_episodes=test_episodes,
    env_config=DEFAULT_ENV_CONFIG,
    algo_config=approx_cfg,
)
approx_time = time.perf_counter() - t0

print(f"Linear Approx Q-learning  —  training time: {approx_time:.2f} s")
print(f"  Feature vector dimension: {int(approx_result['feature_dim'])}")
bm = approx_result['best_test_metrics']
print(f"  Best checkpoint (epoch {int(approx_result['best_epoch'])}):")
print(f"    avg_return = {bm['avg_episode_return']:.4f}")
```

```
print(f"    energy     = {bm['avg_step_energy']:.4f}")
print(f"    latency    = {bm['avg_step_latency']:.4f}")
print(f"    miss_rate  = {bm['miss_rate']:.4f}")
```

```
Linear Approx Q-learning  —  training time: 35.31 s
  Feature vector dimension: 66
  Best checkpoint (epoch 260):
    avg_return = -55.9235
    energy     = 0.1898
    latency    = 0.0074
    miss_rate  = 0.0000
```

In [8]:

```
# Compact training summary (skip extra in-notebook learning-curve plot)
tab_hist = tabular_result['history']
apx_hist = approx_result['history']

best_baseline_return = max(v['avg_episode_return'] for v in baseline_results
best_tab = max(h['eval_test_avg_return'] for h in tab_hist)
best_apx = max(h['eval_test_avg_return'] for h in apx_hist)

print('Training summary (single-seed demo):')
print(f"  Best baseline return : {best_baseline_return:.3f}")
print(f"  Best tabular return  : {best_tab:.3f}")
print(f"  Best approx return   : {best_apx:.3f}")
print(f"  Training time (s)    : tabular={tabular_time:.2f}, approx={approx_
```

```
Training summary (single-seed demo):
  Best baseline return : -57.289
  Best tabular return  : -56.217
  Best approx return   : -55.923
  Training time (s)    : tabular=6.72, approx=35.31
```

6) Reload Saved Best Models

Load saved best artifacts from the full experiment and evaluate them on the same test split.

In [9]:

```
from q_learning import evaluate_q_policy, greedy_action as tab_greedy
from approx_q import LinearQApproximator, evaluate_linear_q_policy

# ── Load pre-trained artifacts from full experiment ──────────────────
TAB_Q_PATH  = ROOT / 'results/tabular_q_learning_120/q_table_best.npy'
APX_W_PATH  = ROOT / 'results/approx_q_learning_120/weights_best.npy'

q_table_best  = np.load(TAB_Q_PATH)
weights_best  = np.load(APX_W_PATH)
approximator  = LinearQApproximator(DEFAULT_ENV_CONFIG)

print("Q-table shape :", q_table_best.shape,
      "  (workload_bins × queue_bins × battery_bins × actions)")
print("Weight vector  :", weights_best.shape,
      f"  ({approximator.base_dim} base features × {approximator.num_actions

# ── Evaluate saved models on test episodes ───────────────────────────
tab_saved = evaluate_q_policy(q_table_best, test_episodes, env_config=DEFAUL
```

```
apx_saved = evaluate_linear_q_policy(weights_best, approximator, test_episoc
                                     env_config=DEFAULT_ENV_CONFIG)

print("\n  Saved best model evaluation on test set:")
print(f"  {'Method':<28}  {'Avg Return':>10}  {'Energy':>7}  {'Latency':>8}
print('  ' + '-' * 65)
for label, m in [('Tabular Q (saved best)', tab_saved), ('Approx Q (saved be
    print(f"  {label:<28}  {m['avg_episode_return']:>10.4f}  "
          f"{m['avg_step_energy']:>7.4f}  {m['avg_step_latency']:>8.4f}  "
          f"{m['miss_rate']:>9.4f}")
```

```
Q-table shape : (5, 5, 5, 3)   (workload_bins × queue_bins × battery_bins ×
actions)
Weight vector  : (66,)   (22 base features × 3 actions)

  Saved best model evaluation on test set:
  Method                        Avg Return   Energy   Latency  Miss Rate
  -----------------------------------------------------------------
  Tabular Q (saved best)          -56.1398   0.1910    0.0065     0.0000
  Approx Q (saved best)           -55.9416   0.1890    0.0087     0.0000
```

Feature Design (Linear Approximation)

`Q(s,a) = w_a^T phi(s)` with action-wise block encoding.

Feature groups:

- bias + continuous terms (`workload` , normalized `queue` , `battery_ratio`)
- one-hot bins for workload/queue/battery
- interaction terms

Total dimension: `66` .

In [10]:
```
# Compact policy snapshot from saved tabular Q-table (no heatmap in report F
action_names = DEFAULT_ENV_CONFIG.action_names
n_w = len(DEFAULT_ENV_CONFIG.workload_bins) - 1
n_q = len(DEFAULT_ENV_CONFIG.queue_bins) - 1
counts = {a: 0 for a in action_names}
for w in range(n_w):
    for q in range(n_q):
        a = int(np.argmax(q_table_best[w, q, 4]))  # battery_bin=4
        counts[action_names[a]] += 1

total = sum(counts.values())
print('Greedy action distribution on (workload_bin, queue_bin), battery_bin=
for k,v in counts.items():
    print(f"  {k:<6}: {v}/{total} ({100*v/total:.1f}%)")
```

```
Greedy action distribution on (workload_bin, queue_bin), battery_bin=4:
  low   : 9/25 (36.0%)
  medium: 10/25 (40.0%)
  high  : 6/25 (24.0%)
```

7) Final Artifact Check

Read final saved summaries and verify artifact paths.

In [11]:
```
# Verify that final artifact files exist
paths = [
    ROOT / 'results/final_figures/final_summary_table.md',
    ROOT / 'results/phase3_multiseed_120.md',
    ROOT / 'results/reward_sensitivity_120.md',
    ROOT / 'results/hyperparam_sensitivity_120.md',
]
for fp in paths:
    print(f"{fp.name}: {'OK' if fp.exists() else 'MISSING'}")
```

```
final_summary_table.md: OK
phase3_multiseed_120.md: OK
reward_sensitivity_120.md: OK
hyperparam_sensitivity_120.md: OK
```

8) Main Results

The table below is the primary comparison under default reward weights `(1.0, 0.6, 2.0)`.

In [12]:
```
summary_md = ROOT / 'results/final_figures/final_summary_table.md'
print(summary_md.read_text(encoding='utf-8'))
```

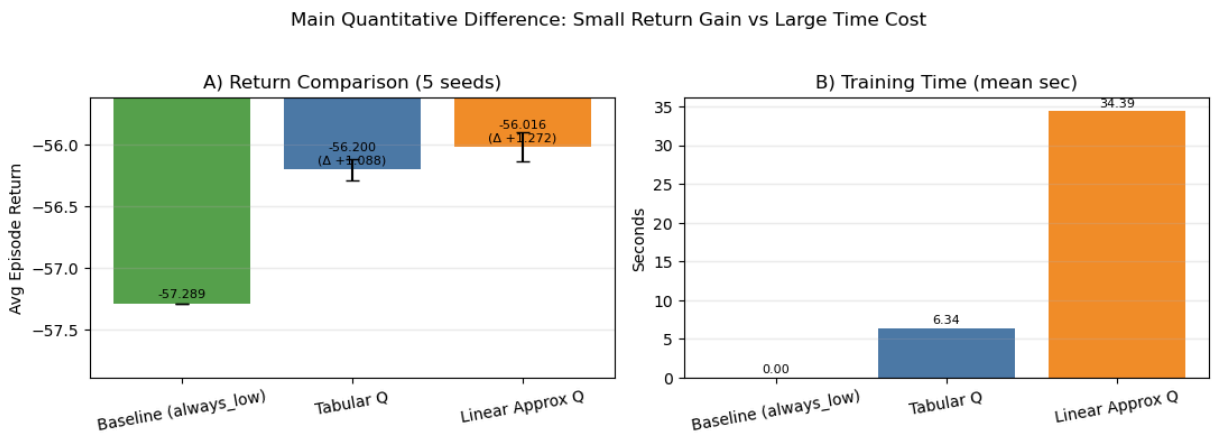
```
# Final Artifact Summary

## Multi-seed Comparison

| Method | Avg Return | Energy | Latency | Miss Rate |
| --- | --- | --- | --- | --- |
| linear_approx_q_learning(best-per-seed) | -56.016 +/- 0.117 | 0.190 +/- 0.
001 | 0.008 +/- 0.001 | 0.000 +/- 0.000 |
| tabular_q_learning(best-per-seed) | -56.200 +/- 0.084 | 0.191 +/- 0.001 |
0.007 +/- 0.001 | 0.000 +/- 0.000 |
| baseline:always_low | -57.289 +/- 0.000 | 0.180 +/- 0.000 | 0.032 +/- 0.00
0 | 0.000 +/- 0.000 |

## Best Reward Setting

- method=approx, (alpha,beta,gamma)=(0.80, 0.60, 2.00), return=-45.098 +/-
0.050

## Best Hyperparameter Setting

- method=approx, (alpha,epsilon_decay,gamma)=(0.030, 0.999, 0.980), return=-
56.017 +/- 0.144

## Generalization Check

- non_overlap: return=-45.348 +/- 0.406 (train/test episodes=20/6)
- overlap_test: return=-45.709 +/- 0.400 (train/test episodes=20/10)
```

Core Figure (How to Read)

Left: average return (higher is better, less negative). Error bars are std across 5 seeds.
Right: mean training wall-clock time. This panel explains the practical cost difference.

In [13]:
```
import json
import matplotlib.pyplot as plt

phase3_json = ROOT / 'results/phase3_multiseed_120.json'
with phase3_json.open('r', encoding='utf-8') as f:
    payload = json.load(f)

rows = payload['rows']
# Keep display order fixed for readability.
order = [
    'baseline:always_low',
    'tabular_q_learning(best-per-seed)',
    'linear_approx_q_learning(best-per-seed)',
]
label_map = {
    'baseline:always_low': 'Baseline (always_low)',
    'tabular_q_learning(best-per-seed)': 'Tabular Q',
    'linear_approx_q_learning(best-per-seed)': 'Linear Approx Q',
}
row_map = {r['method']: r for r in rows}
ordered = [row_map[k] for k in order]

labels = [label_map[r['method']] for r in ordered]
returns = [float(r['avg_episode_return_mean']) for r in ordered]
ret_std = [float(r['avg_episode_return_std']) for r in ordered]
times = [float(r['training_wall_time_sec_mean']) for r in ordered]

baseline_ret = returns[0]
gaps = [x - baseline_ret for x in returns]

fig, axes = plt.subplots(1, 2, figsize=(11, 3.8))
colors = ['#59a14f', '#4e79a7', '#f28e2b']

# Panel A: return comparison (zoomed y-range for visibility)
ax = axes[0]
bars = ax.bar(labels, returns, yerr=ret_std, capsize=4, color=colors)
ax.set_title('A) Return Comparison (5 seeds)')
ax.set_ylabel('Avg Episode Return')
ax.grid(axis='y', alpha=0.25)
ax.set_ylim(min(returns) - 0.6, max(returns) + 0.4)
for i, (b, r, g) in enumerate(zip(bars, returns, gaps)):
    txt = f'{r:.3f}' if i == 0 else f'{r:.3f}\n(Δ {g:+.3f})'
    ax.text(b.get_x() + b.get_width()/2, r + 0.03, txt, ha='center', va='bot
ax.tick_params(axis='x', rotation=10)

# Panel B: training time
ax2 = axes[1]
bars2 = ax2.bar(labels, times, color=colors)
ax2.set_title('B) Training Time (mean sec)')
```

```
ax2.set_ylabel('Seconds')
ax2.grid(axis='y', alpha=0.25)
for b, t in zip(bars2, times):
    ax2.text(b.get_x() + b.get_width()/2, t + 0.3, f'{t:.2f}', ha='center',
ax2.tick_params(axis='x', rotation=10)

fig.suptitle('Main Quantitative Difference: Small Return Gain vs Large Time
plt.tight_layout()
plt.show()

print('Key numbers:')
print(f"  Baseline return      : {returns[0]:.3f}")
print(f"  Tabular Q return     : {returns[1]:.3f} (Δ {gaps[1]:+.3f} vs basel
print(f"  Approx Q return      : {returns[2]:.3f} (Δ {gaps[2]:+.3f} vs basel
print(f"  Training time ratio  : approx/tabular = {times[2]/times[1]:.2f}x")
```




```
Key numbers:
  Baseline return      : -57.289
  Tabular Q return     : -56.200 (Δ +1.088 vs baseline)
  Approx Q return      : -56.016 (Δ +1.272 vs baseline)
  Training time ratio  : approx/tabular = 5.43x
```

9) Sensitivity and Robustness

- Reward weights affect absolute return scale, so comparisons should be made within each setting.
- Hyperparameter sensitivity is moderate: top configurations are close in performance.
- Under stricter deadline thresholds, miss rate becomes informative (not always zero).

10) Expected vs Observed

Observed outcomes match expectations:

- RL methods beat the strongest baseline, with modest margin.
- Approx-Q is slightly better in return.
- Tabular Q is much faster in training time.
- Multi-seed variance is low.

11) Challenges and Future Work

Main challenges: state discretization trade-offs, reward calibration, and limited stress under default deadline.

Future work: multi-device settings, stricter constraints, broader workloads, and stronger policy classes.

12) Conclusion

Both RL methods outperform the best baseline on the default objective. Approx-Q gives slightly better return; tabular Q gives much lower training cost.

The project provides a reproducible simulator, consistent baseline-vs-RL comparison, and clear sensitivity analysis.